Lab Tutorial: Express Application to Create REST API (2627 T1)

Prepared by Dr.Nadeem Qazi

CN5006

In this exercise, you will explore how **REST APIs** are implemented using the **Express.js** framework. The lab builds on the MongoDB CRUD functions developed in your Week 4 lab and demonstrates how these functions can be accessed through REST API routes.
By the end of this lab, you will be able to:

1. Create a basic Express.js web application using Node.js.
2. Create REST API routes using the **GET**, **POST**, **PUT**, and **DELETE** methods.
3. Pass and retrieve URL parameters using **`req.params`**.
4. Build REST API URLs with and without parameters.
5. Create and use JSON files as a simple data source.
6. Read JSON data using Node.js's built-in **fs** module.
7. Connect Express.js applications to MongoDB using Mongoose.
8. Reuse the CRUD functions developed in **Week4Lab.js** and expose them through REST API routes.
9. Test REST APIs using a web browser and HTML forms.
10. Return data to clients in JSON format using **`res.json()`**.

Files Provided for This Lab

This lab tutorial includes the following files:

1. **StudentInfo.html**
2. **PersonForm.html**
3. **Students.json**
4. **Week4Lab.js**
5. **index.js . Note it is your index file for Week5**

The easiest way to complete this exercise is to download all of the supplied files into a single project directory.

1. Open **Visual Studio Code**.
2. Open the folder containing all the downloaded files.
3. Open i**ndex.js**.
4. Install the required Node.js packages if necessary.
5. Run the application using: Week5index.js.

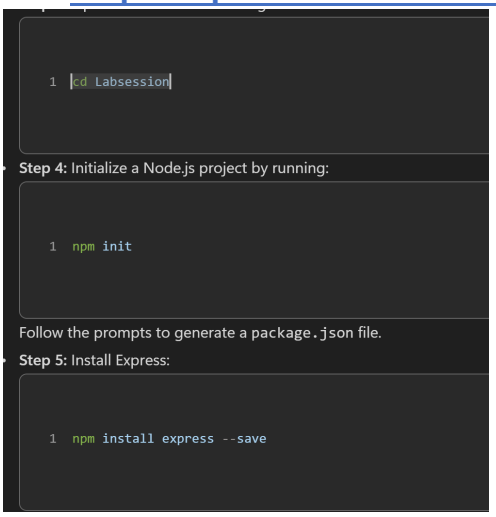
Note you must change the MONGO_URI in Week4lab.js as per your need.

```
//const MONGO_URI = 'mongodb://localhost:27017/Week8';
//if you are using the Mongodb atlas then use this
const MONGO_URI
='mongodb+srv://Youruser:yourpassword@cluster0.tqdie1c.mongodb.net/Employee'
```

Setting Up an Express App in Node.js

- Step 1: Open Visual Studio Code.
- Step 2: Create a folder named Labsession (or any name of your choice).

Step 3: Open the terminal and navigate into the folder:



Lets start it: before you write the code please note // is used to write comment in JS and it is not treated as CODE

Exercise 1: write the code for the basic **express hello application** . NodeJS Core module used: “express”

```
const express = require('express');
const fs= require('fs')
const app = express();

// Built-in middleware for parsing form data and JSON
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Home route
app.get('/', (req, res) => {
    res.send('Hello! This is my first Express application.');
});

// Start server
const PORT = 5000;

app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
});
```

How to Run:

To save and run your Express app:

- Click **File > Save** in Visual Studio Code to save your `index.js` file.
- Open the **Terminal** in VS Code and run the following command to start your app

node index.js

You will see a console message : server is running on port 5000.
Now open Google Chrome or any other browser, and in the location bar of the browser, type this localhost:5000
This is the IP address of the application that we have just created, and 5000 is the port number. In the browser window, you will see the following message
Hello! This is my first Express application.

Exercise 2:

To add more routes to your Express app:

- Return to **Visual Studio Code**.
- Press **Ctrl + C** in the terminal to stop the running application.
- Open your index.js file and add the following code to define two additional routes:

Add two more URLs in the index.js using the following code:

```
app.get('/about', function (req, res) {
  res.send('This is a basic Express application');
});

app.get('/users/:userId/books/:bookId', function (req, res) {
  res.send(req.params);
});
```

To run your updated Express application:

- Save all changes in Visual Studio Code.
- Open the **Terminal** window.
- Run the application again by typing: node index.js

Now open the browser and test following URLs

1. **Localhost:5000**
2. **Localhost:5000/about**
3. **Localhost:5000/users/33/books/123**

Report back what you observed, also try different userids and bookid such as

1. Localhost:5000/users/334/books/143 Note it can be any number.
4. Also, try this Localhost:5000/user/33/books/123: Observe and report what you find on the browser

Now, once again, switch back to Visual Studio Code and press Ctrl-C to terminate the running application. write the following code in Student.json for Exercise 3.

Exercise 3

Step 1: Create a file name Student.json and add the following code to it:

```
{
    "Student1" : {
    "name" : "Jonhthon",
    "Age" : "33",
    "Qualification" : "BSC",
    "Email": "std123@gm.com", "id": 1
    },
    "Student2" :
    { "name" : "David",
    "Age" : "23",
    "Qualification" : "HNC", "Email": "Abc@gm.com", "id": 2
    },
    "Student3" :
    { "name" : "Emily",
    "Age" : "25",
    "Qualification" : "A-level", "Email": "email@gm.com", "id": 3
    }
}
```

Step 2: To read a JSON file in your Express web application:

- **Open your index.js file and write following code**

Note in this code, you will be using __dirname. The __dirname in a node script returns the path of the folder where the current JavaScript file resides. __dirname is used to get the directory name of the currently executing file.

```
app.get('/GetStudents', (req, res) => {
    fs.readFile('./Students.json', 'utf8', (err, data) => {

        if (err) {
            return res.status(500).json({
                status: false,
                statusCode: 500,
                message: 'Error reading Students.json'
            });
        }

        res.status(200).json({
            status: true,
            statusCode: 200,
            requestedAt: new Date().toISOString(),
            requestUrl: req.originalUrl,
            requestMethod: req.method,
            studentData: JSON.parse(data)
        });
    });
});
```

Save all the files and Run the application again. use node index.js

Open the browser window and observe and report what happens when you type the following

Localhost:5000/GetStudents

Report the output of your browser window. What did you see when you ran this URL?

Step 3 Searching the JSON File :Press ctrl C and add more code in the index.js

```
app.get('/GetStudent/:id', (req, res) => {

    console.log("GetStudent called");

    fs.readFile(__dirname + "/Students.json", "utf8", (err, data) => {

        if (err) {
            return res.status(500).json({
                status: false,
                statusCode: 500,
```

```
                message: "Error reading Students.json"

            });
        }

        const students = JSON.parse(data);
        const student = students["Student" + req.params.id];

        console.log(student);

        if (student) {
            res.status(200).json(student);
        } else {
            res.status(404).json({
                status: false,
                statusCode: 404,
                message: "Student Not Found",
                requestedAt: new Date().toISOString(),
                requestUrl: req.originalUrl,
                requestMethod: req.method
            });
        }
    });
});
```

Run and Report

Save all the files and write node index.js and after you received server is running message open the browser and type following URL

- Localhost:5000/ GetStudentid/1 and report what you observed
- Localhost:5000/ GetStudentid/2 and report what you observed
- Localhost:5000/ GetStudentid/3 and report what you observed
- Localhost:5000/ GetStudentid/4 and report what you observed

Report what did you observe for following GetStudent/1, GetStudent/2,GetStudent/4, one by one and explain your output

Once again, press Ctrl-C to terminate the running application and follow these steps in Visual Studio Code.

Exercise 3

Use of Post Method

Create an HTML file **name it as StudentInfo.html,** and write the following code

E**ither you can type it or you can download this file from week 4 and copy it into the same folder**

where you wrote today's code.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
    <meta charset="utf-8" />
    <title>Student Enrollment Form</title>
</head>
```

```
<body bgcolor= beige>
    <form action="/submit-data" method="post" >
<p><Strong> Student Details </p>
        First Name: <input name="firstName" type="text" /> <br/><br/>
        Last Name : <input name="lastName" type="text" /> <br/><br/>
        Email:         <input type="email" id="emailid"
name="email"/><br/> <br/>
        Age :     <input name="myAge" type="text" /> <br/>

 <p>Please select your gender:</p>

  <input type="radio" id="male" name="gender" value="male">
  <label for="male">Male</label><br>
  <input type="radio" id="female" name="gender" value="female">
  <label for="female">Female</label><br>
  <input type="radio" id="other" name="gender" value="other">
  <label for="other">Other</label>

<p> Qualifications </p>
<input type="checkbox" name="Qual" value="GCSE"> GCSE <br/>
  <input type="checkbox" name="Qual" value="A-level"> A- level <br>
  <input type="checkbox" name="Qual" value="Higher National Certificate">
Higher National Certificate/Level 4 <br/>
  <input type="checkbox" name="Qual" value="HND"> Foundation
Degree/HND/DipHE/Level 5 <br/>
  <input type="checkbox" name="Qual" value="B.Sc"> Bachelor Degree/Graduate
diploma or
Certificate/Level 6 <br/>

  <input type="checkbox" name="Qual" value="Master Degree"> Master
Degree/PGCE/Level7 <br/>
  <input type="checkbox" name="Qual" value="PhD"> PhD/Level8 <br/>
 <input type="submit" />
    </form>
</body>
</html>
```

Step 2 append following method in index.js

```
// showing html form
app.get('/studentinfo', (req, res) => {
    res.sendFile('StudentInfo.html', { root: __dirname });
});

app.post('/submit-data', (req, res) => {

    const name = req.body.firstName + ' ' + req.body.lastName;
    const age = req.body.myAge
    const Gender = ' Gender: ' + req.body.gender;
```

```
    const qual = 'Qualification: ' + req.body.Qual;

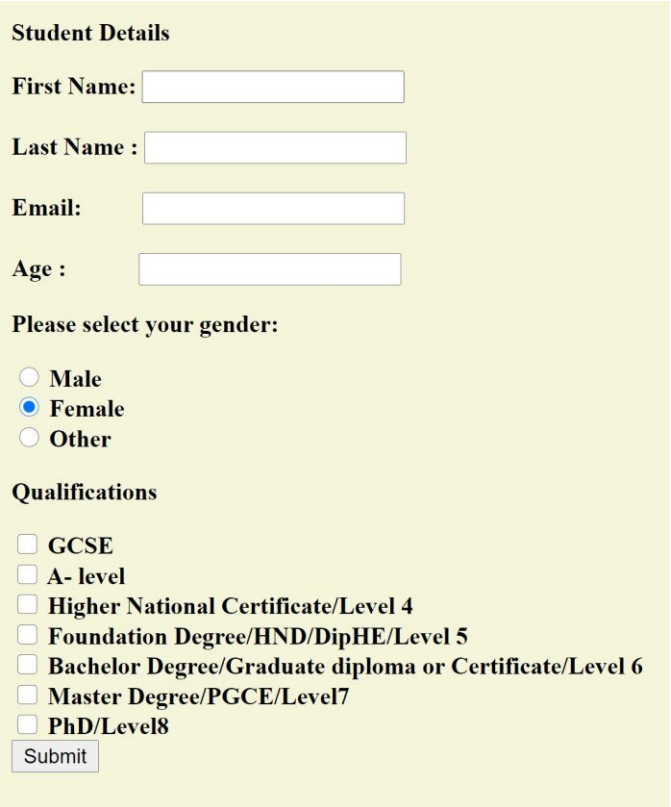
    console.log(req.body.Qual);

    res.send({
        status: true,
        message: 'Form Details',
        data: {
            name: name,
            age: age,
            gender: Gender,
            qualification: qual
        }
    });
});
```

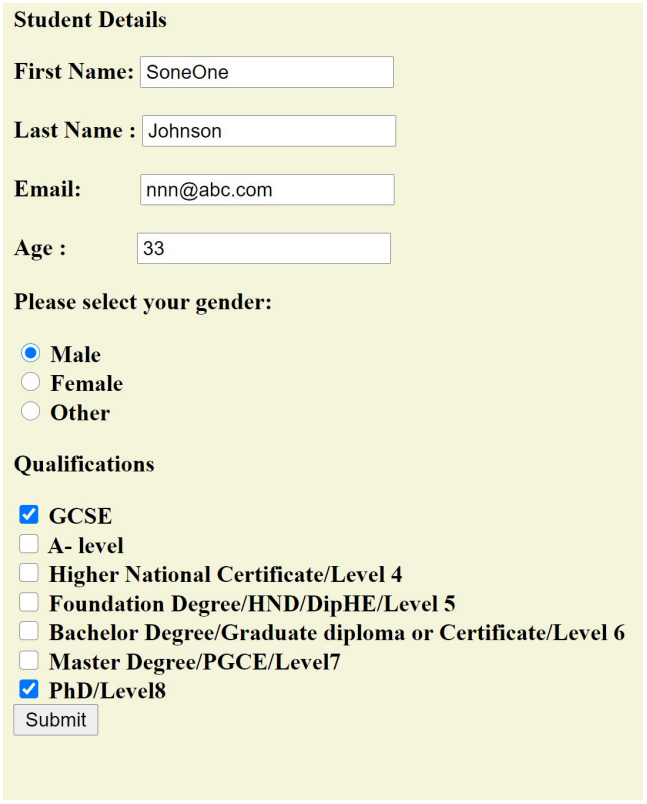
Save all the files and write node index.js to start the application then open the browser window and type

Localhost:5000/ studentinfo

You will observe the following form



Fill this form with appropriate dummy data like this



Submit this form by clicking the submit button

You may see the following output based on what you entered.:

```
{"status":true,"message":"form Details","data":{"name":"SoneOne Johnson
","age":"33 Gender: male ","Qualification":" QualificationGCSE,PhD"}}
```

Calling MongodB Database functions in to REST API

Last week, you created functions for performing **CRUD operations in MongoDB** using **Mongoose**, including inserting, deleting, updating, and retrieving records. You connected your application to MongoDB either through **MongoDB Atlas** or a local MongoDB installation on your laptop.

We will use that code for crude operation, provided in the file **`Week4Lab.js`** for today's lab. Download this file and place it in the same directory where you created **`index.js`**. Note this file has exported the functions using the command :

```
module.exports = {
    insertOne,
    insertMany,
    findAll,
    findFemalesOver30,
    countDocuments,
    deleteManyPersons,
    updateManyPersons
};
```

Next, append the following code to your **`index.js`** file. You may add this code either:

- At the beginning of `index.js`, or
- Immediately after the code you completed in **Exercise 3**.

This will allow your application to use the Mongoose connection and query functions provided in `Week4Lab.js`.

```
const mongoose = require('mongoose');
const {
    insertOne,
    findAll,
    countDocuments,
    findFemalesOver30,
    deleteManyPersons,
    updateManyPersons
} = require('./Week4Lab');
const MONGO_URI =
'mongodb+srv://nhqbrunellabtop_db_user:Hhqazi2906@cluster0.tqdie1c.mongodb.net/E
mployee';

mongoose.connect(MONGO_URI)
    .then(() => console.log('Connected to MongoDB'))
    .catch(err => console.log(err));
```

Now run the index.js , if every thing runs fine you will see Connected to MongDB in console window

Create REST APIs

We will now create a **POST REST API** to insert a new record into the **Employee** database that you created in Week 4. Unlike a GET request, which is used to retrieve data, a **POST request** is used to send data from the client to the server. In this case, the POST request will collect employee information entered by the user, such as **name, age, gender, and salary**, and then insert this information into the MongoDB database.

To collect the required information from the user, we will create an **HTML form**. The code for this form is provided on Moodle in a file called **PersonForm.html**. Download this file and save it in the same directory as your **index.js** file. In **PersonForm.html**, the following form tag is defined:

```
    <form action="/insertOne" method="POST">
```

The **action** attribute specifies the URL of the POST route that will handle the submitted data. In this example, the route is **/insertOne**. When the user completes the form and clicks the **Submit** button, a POST request is sent to the server and the **insertOne** function is executed.

The complete code for the **insertOne** route is provided below. Copy this code into your **index.js** file. Once implemented, users will be able to enter employee details through the form and store them directly in the Employee database. Note you need to run the RESTAPI Route ShowForm,which will display the form for userinput and when you press the submit button you will be able to call post method.

```
app.post('/insertOne', async (req, res) => {

    try {

        const result = await insertOne(
            req.body.name,
            req.body.age,
            req.body.gender,
            req.body.salary
        );

        res.status(201).json({
            status: true,
            message: "Person inserted successfully",
            data: result
        });

    } catch (err) {

        res.status(500).json({
            status: false,
            message: err.message
        });

    }
});
app.get('/Showform', (req, res) => {

    res.sendFile('Personform.html', {
        root: __dirname
    });

});
```

Find ALL Records RESTAPI:

We will now create a GET REST API to retrieve all person records from the Employee

database. To achieve this, we will use the findAll() function defined in Week4Lab.js. Recall that this function uses MongoDB's find() method to retrieve all documents from the collection.

Copy the following code into your index.js file:

```
app.get('/persons', async (req, res) => {
    try {
        console.log("calling all find")
        const result = await findAll();
        res.json(result);
        console.log("result")
    } catch (err) {
        res.status(500).json(err);
    }

});
```

Note that the **`findAll()`** method retrieves all records from the Employee database. The returned records are stored in the **`result`** variable and sent to the browser in JSON format using:
localhost:5000/persons
To test the API, open a browser and navigate to: http://localhost:5000/persons
The browser will display all employee records currently stored in the **Employee** database collection. This confirms that the GET REST API is working correctly and is successfully retrieving data from MongoDB.
The compete code of this Week exercise is provided as Week5index.js, you can also download it and use it directly.

Portfolio Activity :

Save your work, write a short reflective report, and upload your source code to **GitHub** as evidence for your portfolio. Add screen shots of the all the RESTAPI in your portfolio. For your portfolio, extend your REST API application by creating additional API endpoints that use the MongoDB functions developed in **Week4Lab.js**, including:

- **insertMany()** - Insert multiple employee records into the database.
- **findFemalesOver30()** - Retrieve all female employees aged 30 or above.
- **countDocuments()** - Return the total number of employee records in the collection.
- **updateManyPersons()** - Update multiple employee records based on a specified condition.
- **deleteManyPersons()** - Delete multiple employee records matching a given condition.

Evidence Required

Include the following in your portfolio submission:

- Screenshots showing the REST APIs running successfully.
- Screenshots of the output displayed in the browser or API client.
- Screenshots of the corresponding records in MongoDB (where appropriate).
- A link to your GitHub repository containing the source code.

Reflection

Write a short reflective report (approximately 150-200 words) describing:

- What you learned about Express.js and REST APIs.
- How Express routes were connected to MongoDB functions.
- Challenges encountered during development and how they were resolved.
- Summary steps of creating RESTAPI URL for either insertMany or **findFemalesOver30()**.